

Contents

AI/ML in Biomarker Discovery	1
miRNA-Based Biomarker Discovery in Alzheimer's Disease	1
6-Week Intensive Course	1
Overview	1
System Requirements	2
Part 1 — Setting Up RStudio	2
Step 1: Install R	2
Step 2: Install RStudio Desktop	2
Step 3: Configure RStudio for This Course	2
Step 4: Set Your Course Working Directory	3
Step 5: Install All R Packages (One-Time, ~15 minutes)	3
Part 2 — Setting Up VS Code for R (Alternative to RStudio)	3
Step 1: Install R (same as above)	4
Step 2: Install VS Code	4
Step 3: Install the R Extension	4
Step 4: Install the <code>languageserver</code> R Package	4
Step 5: Configure VS Code R Settings	4
Step 6: Open a Terminal and Run R	4
Part 3 — Setting Up Python (for Lab 4B)	4
Step 1: Install Anaconda	5
Step 2: Create the Course Environment	5
Step 3: Launch JupyterLab	5
Step 4: Verify Python Packages	5
Part 4 — Project Directory Structure	5
Week-by-Week Plan	6
Week 1 — Foundations: Biology & Analytical Tools	6
Week 2 — Data Acquisition & Quality Control	6
Week 3 — Exploratory Data Analysis	6
Week 4 — Feature Selection & Classical Machine Learning	7
Week 5 — Advanced ML & External Validation	7
Week 6 — Biological Interpretation & Clinical Translation	8
Troubleshooting	8
Quick Reference: Script Execution Order	9

AI/ML in Biomarker Discovery

miRNA-Based Biomarker Discovery in Alzheimer's Disease

6-Week Intensive Course

Overview

This course teaches wet-lab biologists how to apply AI/ML techniques to discover and validate blood-based miRNA biomarkers for Alzheimer's disease (AD). No prior coding experience is required. By Week 6 you will have built, evaluated, and biologically interpreted a miRNA classifier trained on real publicly available clinical datasets.

Disease focus: Alzheimer's Disease (AD)

Biomarker class: Circulating microRNA (miRNA) from blood

Datasets: GSE120584 (serum RNA-seq, 148 samples) and GSE46579 (whole blood microarray)

Languages: R (Bioconductor) for data processing and statistics; Python (scikit-learn) for machine learning

All course materials, lecture write-ups, and R lab templates are available on GitHub:
<https://github.com/ricchasethi/ML-based-biomarker-discovery/tree/main>

System Requirements

Component	Minimum	Recommended
RAM	8 GB	16 GB
Storage	10 GB free	20 GB free
OS	Windows 10, macOS 11, Ubuntu 20.04	Any current version
Internet	Required for GEO downloads (Week 2)	Stable broadband

Part 1 — Setting Up RStudio

RStudio is the primary IDE for Weeks 1–6. All R scripts in this course are designed to be run interactively inside RStudio.

Step 1: Install R

1. Go to <https://cran.r-project.org/>
2. Click your operating system (Windows / macOS / Linux)
3. Download and run the installer for the **latest R version (4.3.0)**
 - Windows: click *base*, then *Download R x.x.x for Windows*
 - macOS: download the `.pkg` file matching your chip (Apple Silicon = `arm64`; Intel = `x86_64`)
 - Ubuntu: `sudo apt install r-base` (version from CRAN, not the default Ubuntu repo)
4. Verify installation: open a terminal and type `R --version` — you should see **R version 4.3.x** or higher

Step 2: Install RStudio Desktop

1. Go to <https://posit.co/download/rstudio-desktop/>
2. Click **Download RStudio Desktop** (free version — “RStudio Desktop Open Source License”)
3. Run the installer. On macOS, drag RStudio to your Applications folder.
4. Launch RStudio. You should see four panes:
 - **Top-left:** Script editor (where you write code)
 - **Bottom-left:** Console (where code runs and output appears)
 - **Top-right:** Environment (shows loaded variables)
 - **Bottom-right:** Files / Plots / Help

Step 3: Configure RStudio for This Course

Open RStudio and apply these settings (menu: **Tools** → **Global Options**):

Setting	Location	Recommended Value
Restore .RData on startup	General → Basic	Unchecked (prevents stale objects)
Save workspace to .RData on exit	General → Basic	Never
Default text encoding	Code → Saving	UTF-8
Soft-wrap R source files	Code → Editing	Checked

Setting	Location	Recommended Value
Rainbow parentheses	Code → Display	Checked (helps with nested code)
Show margin at column	Code → Display	80

Step 4: Set Your Course Working Directory

Every R script in this course reads and writes files relative to a single **course root folder**. Set this once at the start of every session:

```
# Replace this path with your actual course folder location
setwd("~/Documents/ML-based-biomarker-discovery")

# Verify it worked
getwd()
```

Tip: In RStudio you can also set this via Session → Set Working Directory → Choose Directory.

Step 5: Install All R Packages (One-Time, ~15 minutes)

Open miRNA/Week1_Setup_Template.R in RStudio and run **Sections 2 and 3** by pressing Ctrl+Enter (Windows/Linux) or Cmd+Enter (Mac) on each line. This installs every package needed across all 6 weeks.

To run manually:

```
# Install BiocManager first (required for all Bioconductor packages)
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

# Bioconductor packages
BiocManager::install(c(
  "GEOquery", "DESeq2", "limma", "edgeR",
  "multiMiR", "miRBaseConverter", "clusterProfiler", "org.Hs.eg.db",
  "affy", "oligo", "sva"
), ask = FALSE, update = FALSE)

# CRAN packages
install.packages(c(
  "ggplot2", "tidyverse", "pheatmap", "reshape2", "RColorBrewer",
  "ggrepel", "gridExtra", "cluster", "factoextra", "car",
  "MASS", "pROC", "knitr", "rmarkdown"
))
```

Verify everything loaded:

```
# Paste this in the Console - if no errors appear, your environment is ready
library(GEOquery); library(DESeq2); library(limma); library(edgeR)
library(multiMiR); library(clusterProfiler); library(pheatmap)
library(ggplot2); library(cluster); library(pROC)
sessionInfo() # save this output - useful for troubleshooting
```

Part 2 — Setting Up VS Code for R (Alternative to RStudio)

If you prefer Visual Studio Code, follow these steps instead of Part 1.

Step 1: Install R (same as above)

Follow Step 1 from Part 1.

Step 2: Install VS Code

1. Go to <https://code.visualstudio.com/> and download the installer for your OS
2. Run the installer with default settings

Step 3: Install the R Extension

1. Open VS Code
2. Click the **Extensions icon** in the left sidebar (or press **Ctrl+Shift+X**)
3. Search for “**R**” and install the extension by **REditorSupport** (publisher: REditorSupport)
4. Also install the “**R Debugger**” extension by **RDebugger**

Step 4: Install the languageserver R Package

This enables autocomplete, hover documentation, and linting inside VS Code:

```
# Run this in your R terminal or RStudio Console
install.packages("languageserver")
install.packages("httpgd") # for interactive plots inside VS Code
```

Step 5: Configure VS Code R Settings

Open Settings (**Ctrl+,**), search for each setting below, and apply it:

Setting	Value
<code>r.term.windows</code>	Path to your R.exe, e.g. <code>C:\Program Files\R\R-4.4.x\bin\R.exe</code>
<code>r.term.mac</code>	<code>/usr/local/bin/R</code> (or <code>/opt/homebrew/bin/R</code> for Apple Silicon)
<code>r.plot.useHttpgd</code>	<code>true</code> (plots appear in the VS Code panel)
<code>r.lsp.enabled</code>	<code>true</code>
<code>r.bracketedPaste</code>	<code>true</code> (enables Ctrl+Enter to send selected lines to terminal)

Step 6: Open a Terminal and Run R

1. Open an integrated terminal: **Ctrl+`** (backtick)
2. Type `R` and press Enter — an interactive R session starts
3. Run code by selecting lines in the editor and pressing **Ctrl+Enter**

Note: RStudio provides a more beginner-friendly experience with a built-in file browser, environment viewer, and plot panel. If you are new to R, RStudio is strongly recommended.

Part 3 — Setting Up Python (for Lab 4B)

Python is used for the machine learning classifiers in Week 4. The recommended approach is Anaconda with a dedicated course environment.

Step 1: Install Anaconda

1. Go to <https://www.anaconda.com/download>
2. Download and install the **Anaconda Individual Edition** for your OS (Python 3.11+)
3. During installation on Windows: check “Add Anaconda to my PATH” (makes the next steps easier)

Step 2: Create the Course Environment

Open a terminal (macOS/Linux) or **Anaconda Prompt** (Windows):

```
conda create -n biomarker_ml python=3.11
conda activate biomarker_ml
pip install pandas numpy scipy statsmodels scikit-learn matplotlib seaborn \
    umap-learn shap jupyterlab
```

You will need to run `conda activate biomarker_ml` at the start of every Python session.

Step 3: Launch JupyterLab

```
conda activate biomarker_ml
cd /path/to/ML-based-biomarker-discovery
jupyter lab
```

JupyterLab opens in your browser. Navigate to the `miRNA/` folder to find lab notebooks.

Step 4: Verify Python Packages

In a new Jupyter notebook, run:

```
import numpy, pandas, sklearn, matplotlib, seaborn, umap, shap, xgboost
print("All packages OK - sklearn version:", sklearn.__version__)
```

Part 4 — Project Directory Structure

Before running any script from Week 2 onward, create this folder structure inside the course root directory. The Week 2 script creates it automatically, but you can also create it manually:

```
ML-based-biomarker-discovery/
├── miRNA/                                ← all course scripts live here
│   ├── Week1_Foundations.md
│   ├── Week1_Setup_Template.R
│   ├── Week2_DataAcquisition_QC.md
│   ├── Week2_DataAcquisition_QC.R
│   ├── Week3_EDA.md
│   ├── Week3_EDA.R
│   ├── Week4_FeatureSelection_ML.md
│   ├── Week4_DE_FeatureSelection.R
│   ├── Week5_AdvancedML_Validation.md
│   ├── Week5_Validation.R
│   ├── Week6_BiologicalInterpretation.md
│   └── Week6_Interpretation.R
├── data/
│   ├── raw/                             ← GEO downloads (never modify these)
│   └── processed/                       ← clean matrices, passed between weeks
├── qc_reports/                          ← QC plots and exclusion logs
└── results/                             ← output plots and tables (Weeks 3-6)
```

Week-by-Week Plan

Week 1 — Foundations: Biology & Analytical Tools

Lecture write-up: Week1_Foundations.md

Lab script: Week1_Setup_Template.R

What you will learn: - Alzheimer's disease pathology: amyloid plaques, tau tangles, neuroinflammation, and the 15–20 year preclinical window - Why current diagnostics (CSF tau, PET imaging) are inaccessible and why blood-based biomarkers are needed - miRNA biogenesis (Drosha → Dicer → RISC), mechanism of gene silencing, and why miRNAs are stable in serum - How to distinguish microarray vs RNA-seq vs RT-qPCR measurement technologies - Key databases: GEO, miRBase v22, ADNI, AMP-AD

Lab tasks: 1. Install R 4.3.0 and RStudio (or VS Code with R extension) 2. Run Week1_Setup_Template.R Sections 2–4 to install and verify all packages 3. Complete the R basics walkthrough (vectors, data frames, ggplot2) 4. Generate your first plot: AD-relevant miRNA expression bar chart 5. Test GEOquery connectivity to NCBI servers

Deliverable: Working R environment with all packages loaded and `sessionInfo()` output saved.

Week 2 — Data Acquisition & Quality Control

Lecture write-up: Week2_DataAcquisition_QC.md

Lab script: Week2_DataAcquisition_QC.R

What you will learn: - How to navigate NCBI GEO and identify usable miRNA datasets - Downloading GSE120584 (serum RNA-seq) and GSE46579 (microarray) programmatically with GEOquery - ExpressionSet structure and metadata extraction - QC for RNA-seq: library size, detected miRNA count, RLE plots - QC for microarray: NUSE, correlation heatmap - Normalisation: DESeq2 VST (RNA-seq) and RMA (microarray) - Hemolysis detection using miR-23a/miR-451a ratio - Batch effect detection (PCA, correlation) and correction (ComBat, removeBatchEffect)

Lab tasks (run in RStudio, all sections in order): 1. Section 1: Create project directory structure 2. Sections 2–4: Download and parse both GEO datasets 3. Sections 5–7: RNA-seq QC and DESeq2 normalisation on GSE120584 4. Sections 8–9: Microarray QC and RMA normalisation on GSE46579 5. Section 10: Batch correction 6. Final: Confirm data/processed/ contains the clean .rds files

Key output files produced: - data/processed/GSE120584_expr_clean.rds — VST-normalised expression matrix - data/processed/GSE120584_metadata_clean.rds — sample metadata - data/processed/GSE120584_counts_ — raw filtered counts for DESeq2 - data/processed/GSE46579_expr_rma.rds — RMA-normalised microarray matrix

Week 3 — Exploratory Data Analysis

Lecture write-up: Week3_EDA.md

Lab script (R): Week3_EDA.R

What you will learn: - Per-miRNA statistics: coefficient of variation, IQR, zero inflation - PCA: covariance matrix intuition, scree plots, PC1/PC2 scatter, biplot, loadings table - Hierarchical clustering with Ward.D2 linkage; reading a dendrogram - Gap statistic and silhouette width for optimal cluster number selection - Cluster purity: quantifying how well unsupervised clusters recover clinical labels - Quantifying confounder effects via partial R^2 (age and sex vs principal components) - Mahalanobis distance for outlier detection

Lab tasks (RStudio, Week3_EDA.R): 1. Sections 1–4: Load data, compute stats, plot zero inflation and density 2. Section 5: Apply IQR variance filter; save variance-filtered RDS 3. Section 6: Run PCA; generate scree plot, PC1/PC2 scatter, biplot, loadings table 4. Sections 7–10: Hierarchical clustering, gap statistic, silhouette, k-means 5. Section 11: heatmap with Group/Sex/Age annotation tracks 6. Sections 12–13: Confounder partial R^2 and Mahalanobis outlier detection

Key output files produced: - data/processed/GSE120584_expr_varianceFiltered.rds — input for Week 4 R script - results/pca_pc1_loadings.csv, results/heatmap_top50_miRNAs.png, and others

Week 4 — Feature Selection & Classical Machine Learning

Lecture write-up: Week4_FeatureSelection_ML.md

Lab script (R): Week4_DE_FeatureSelection.R

Lab 4B (Python): Week4_ML_Classifier.ipynb — reads GSE120584_expr_forML.csv

What you will learn: - Why no single miRNA is sufficient and why a panel + ML is needed - DESeq2 workflow: negative binomial GLM, size factor normalisation, dispersion estimation, Wald test - LFC shrinkage (lfcShrink): why it matters for noisy, low-count miRNAs - Three-way disease comparisons: AD vs Control, MCI vs Control, AD vs MCI — and what each means biologically - limma-voom pipeline for the microarray validation dataset - Volcano plots and MA plots: how to read them and what artefacts to look for - Three classes of feature selection: filter (Mann-Whitney U), wrapper (RFE), embedded (LASSO, RF importance) - Building logistic regression, SVM (RBF kernel), and random forest classifiers - Model evaluation: confusion matrix, sensitivity, specificity, PPV, NPV, AUC, precision-recall curves - Why AUC alone is insufficient for imbalanced clinical datasets

Lab 4A tasks (RStudio, Week4_DE_FeatureSelection.R): 1. Sections 2–5: DESeq2 setup, run, LFC shrinkage, volcano and MA plots for all three comparisons 2. Section 6: limma pipeline on GSE46579 3. Section 7: Cross-dataset overlap analysis (DESeq2 limma) 4. Section 8: Mann-Whitney U filter feature selection 5. Section 9: Build consensus feature table 6. Section 10: Export expr_forML.csv and labels_binary.csv for Python Lab 4B

Lab 4B tasks (JupyterLab, Week4_ML_Classifier.ipynb): 1. Load exported feature matrix and binary labels 2. Apply Mann-Whitney filter; perform 80/20 stratified train/test split 3. Fit logistic regression, SVM (with grid search), and random forest 4. Generate confusion matrices, ROC curves, precision-recall curves 5. Run 5-fold cross-validation; report mean AUC $\pm$ 95% CI per model 6. Extract random forest feature importances; look up top 3 in miRTarBase

Key output files produced: - results/de_results_deseq2_AD_vs_Control.csv (and MCI vs Control, AD vs MCI) - results/consensus_features_Week4.csv — starting point for Week 5 nested CV - data/processed/GSE120584_expr_forML.csv, data/processed/GSE120584_labels_binary.csv

Week 5 — Advanced ML & External Validation

Lecture write-up: Week5_AdvancedML_Validation.md

Lab script: Week5_Validation.R

What you will learn: - Bias-variance tradeoff and the five most common forms of data leakage in bioinformatics - Nested cross-validation with caret: unbiased AUC estimation and hyperparameter tuning - Two classifiers in R: Random Forest (randomForest) and LASSO logistic regression (glmnet) - SHAP feature importance in R using the fastshap package; beeswarm summary plot - Three-class classification (AD / MCI / Control): multiclass metrics and confusion matrices - Cross-platform harmonization: miRBaseConverter, MIMAT accession mapping, z-score standardisation - DeLong's test for AUC comparison between training CV and external validation (pROC package) - Calibration plots and Brier score: does the model's probability output mean what it says? - TRIPOD and STARD reporting guidelines for biomarker ML studies

Lab tasks (RStudio, Week5_Validation.R, all sections in order): 1. Section 2: Load both datasets; harmonize miRNA names via miRBaseConverter 2. Section 3: Find feature intersection between platforms 3. Section 4: Apply per-dataset z-score standardisation; save harmonized RDS objects 4. Section 6: Train Random Forest and LASSO via `caret` nested CV; extract fold predictions 5. Section 7: Compute SHAP values with `fastshap`; save `shap_feature_importance.csv` 6. Section 8: Apply models to GSE46579; DeLong’s test (training CV AUC vs external AUC) 7. Section 9: Calibration plots and Brier score 8. Section 10: Compile and print model results summary table

Key output files produced: - `data/processed/harmonized_expr.rds`, `data/processed/metadata_harmonized.rds`
- `results/Week5/shap_feature_importance.csv` (read by Week 6) - `results/Week5/roc_curves.png`,
`results/Week5/model_performance_summary.csv`

Week 6 — Biological Interpretation & Clinical Translation

Lecture write-up: `Week6_BiologicalInterpretation.md`

Lab script: `Week6_Interpretation.R`

What you will learn: - How to integrate SHAP importance scores with DE fold changes into a composite biomarker ranking - Querying 14 miRNA–target databases simultaneously with `multiMiR`; filtering for validated interactions - KEGG pathway enrichment analysis (does the AD pathway `hsa05010` appear enriched?) - GO Biological Process enrichment and redundancy removal with `clusterProfiler` - Generating the “paper figure”: a forest plot combining `log2FC`, SHAP importance, and key targets - Regulatory considerations for a blood-based miRNA diagnostic: FDA IVD pathway overview

Lab tasks (RStudio, Week6_Interpretation.R, all sections in order): 1. Section 2: Load SHAP results and DE tables; build composite ranking score; select top 15 miRNAs 2. Section 3: `multiMiR` target query; filter for validated targets; cross-reference known AD genes 3. Section 4: KEGG enrichment — confirm or deny `hsa05010` enrichment 4. Section 5: GO enrichment with term simplification 5. Section 6: Generate forest plot (the “main figure”) 6. Section 7: Print the full 6-week pipeline summary table

Key output files produced (in `results/Week6/`): - `composite_biomarker_ranking.csv`, `validated_targets.csv`
- `kegg_enrichment.csv`, `go_enrichment_simplified.csv` - `biomarker_forest_plot.png`

Troubleshooting

“there is no package called ‘X’”

Run `BiocManager::install("X")` (Bioconductor) or `install.packages("X")` (CRAN), then try again.

“package ‘X’ was built under R version Y.Z”

This warning is usually harmless. Continue unless you see an actual error downstream.

“Error in `getGEO` ... could not resolve host”

No internet connection, or NCBI servers are temporarily down. Check your connection and try again in a few minutes. On institutional networks a VPN may be required.

“‘BiocManager’ is not available for R version X.Y.Z”

Your R version is below 4.3.0. Download the latest R from <https://cran.r-project.org/> and reinstall RStudio after upgrading.

DESeq2 or limma gives different results than expected

Confirm your working directory is the course root and that all `data/processed/` files from the previous week exist. Each script depends on the outputs of all prior scripts.

Jupyter notebook cannot import `umap` or `shap`

Confirm your `biomarker_ml` conda environment is activated before launching JupyterLab: `conda activate biomarker_ml && jupyter lab`.

Quick Reference: Script Execution Order

Week 1: <code>Week1_Setup_Template.R</code>	← one-time package installation
Week 2: <code>Week2_DataAcquisition_QC.R</code>	← downloads data; run with internet access
Week 3: <code>Week3_EDA.R</code>	← complete EDA in R
Week 4: <code>Week4_DE_FeatureSelection.R</code>	← then open <code>Week4_ML_Classifier.ipynb</code>
Week 5: <code>Week5_Validation.R</code>	← complete ML pipeline in R
Week 6: <code>Week6_Interpretation.R</code>	← final week; reads all prior outputs

Each script ends with a summary printout listing every file it produced and what to open next.